Supplementary Material: Extending Mined Rules for Link Prediction with Schema-based Exceptions

Overview

This document provides additional resources to support reproducibility and reuse of the artifacts associated with the paper.

1 Code Repository

Main repository: https://github.com/Zamprognog/NMRuleExtension.git
Artifact/Zenodo snapshot: 10.5281/zenodo.22125113

1.1 Repository structure

- `src/main/java/nmRuleExtension/`: implementation (grounding engines, rule parsers, materialization, evaluation)
- `scripts/`: experiment driver (`run_experiments.sh`) and result collection (`collect_results.sh`)
- `data/[dataset]/`: per-dataset configuration JSON, splits, schema, mined rules and prediction outputs (downloaded from the Zenodo snapshot above)
- `runs/[timestamp]/`: generated logs, `lp_results.csv`, `mat_results.csv` and `RESULTS.md`
- `tools/`: rule miners (AMIE, AnyBURL), their per-dataset configurations, and preprocessing notebooks

1.2 Environment and installation

- OS tested: macOS 14.6.1 , Rocky Linux 8.10
- Language/runtime: Java 21, Maven 3.6+
- Dependencies: resolved by Maven (`mvn clean install`); see `pom.xml`
- Triple store used for the materialization evaluation: GraphDB (batch) and Apache Jena (in-run)
- Installation instructions: see `README.md` in the repository.

2 Datasets

The four KGs used in the experiments are listed below, followed by the derived variant built for the inverse-functionality analysis. Each one is driven by a flat configuration JSON at `data/[dataset]/[dataset].json`, pointing at the splits, the schema file, the entity types file and the four mined rule sets. The prepared versions of all four are contained in the Zenodo snapshot; the pointers below identify the original sources and the preprocessing applied to them. Manual preparation steps carried out *after* download (i.e. steps that are not reproducible by re-downloading) are logged in `data/DATA_PREPARATION.md`.

Nell995

– Schema and ontology reachable here

CSKG2.0

– Dataset and benchmark reachable here
– Schema reachable here

Hetionet

– Splits are generated from the `pykeen.datasets.hetionet` dataset
– Metaedges from hetionet release

YAGO4.5-10

– Data from the official release
– Preprocessing:
  • `yago-disjoints.ttl` manually extracted from `yago-schema.ttl`
  • `properties_d_r_f.csv` manually extracted from `yago-schema.ttl`
  • Convert `yago-facts.hdt` into `.nt` (rdf2hdt library)
  • Remove all literals and triples not relevant for LP via `tools/filterOut_literals`, obtaining `yago4.5_triples.txt` (or `.nt`)
  • Run `filter_ntfile.ipynb` to obtain `yago4.5-10`

YAGO4.5-10-flip

Derived variant of YAGO4.5-10 used only for the inverse-functionality analysis of Section 5.1.1. Built by `yago_flip_experiment/flip_yago.py` from the prepared YAGO4.5-10 files, which are left untouched.

– Flipped properties: `schema:birthPlace` (140,221 triples) and `schema:deathPlace` (55,941).

- Data: every (s, r, o) is rewritten as (o, r^{-1}, s) in the train/valid/test TSVs and in the full `.nt` graph. Split sizes are unchanged (3,222,052 / 16,273 / 16,273) and the entity types file is copied verbatim
- T-Box: the axioms of the original property are dropped; the inverse is declared `rdf:Property` and `owl:InverseFunctionalProperty` with *domain and range swapped* with respect to the original.

3 Rule miners and configurations

- **AnyBURL**: 23-1 (`tools/AnyBURL_og.jar`). Per-dataset property files are in `tools/setup/[dataset]/`, with `[dataset]` one of `NELL`, `YAGO4.5`, `CSKG`, `hetionet`. Two rule sets are mined per dataset with a 100s snapshot, confidence threshold 0.1 and at least 2 correct predictions: `config-learn.properties` for the full set (cyclic, acyclic and grounded rules, all of maximum length 3) and `config-learn-onlyCP.properties` for the closed-path-only set (`MAX_LENGTH_ACYCLIC` and `MAX_LENGTH_GROUNDED_CYCLIC` set to 0).
- **AMIE3**: 3.5.1 (`tools/amie3.5.1.jar`), command lines used can be found in `tools/amie_calls.txt` and `tools/amiemine.sh`. Rules are mined twice per dataset, with `-maxad 3` and `-maxad 4`, in both cases with `-mins 10 -minpca 0.1 -minc 0.1 -full`. As discussed above, the `-const` variant (pattern rules with constants) is not used in the experiments.

4 Reproducing the experiments

1. Install dependencies and build (Section 1.2): `mvn clean install`.
2. Download the datasets (Section 2) and extract them into `data/`.
3. Run the pipeline. The driver script builds the project, runs the requested stages over the requested datasets, tees each stage's output to `runs/[timestamp]/` and collects the metric tables into `lp_results.csv`, `mat_results.csv` and `RESULTS.md`.

```
# all stages (lp = link prediction, mat = materialization,
# mateval = violation counting) over the 3 default datasets
# (nell, yago, cskg; hetionet is opt-in)
./scripts/run_experiments.sh

# a subset; dataset keys: nell, yago, cskg, hetionet
./scripts/run_experiments.sh --datasets hetionet --stages lp

# JVM heap for the stages (default 12g; YAGO4.5-10 needs the most)
./scripts/run_experiments.sh --heap 16g

# materialization rule-confidence thresholds (defaults to 1,5,10)
./scripts/run_experiments.sh --stages mat,mateval \
    --percentages 1,5,10
```

```
# print the plan without running anything
./scripts/run_experiments.sh --dry-run
```

The individual entry points can also be called directly; each takes a dataset configuration JSON as its first argument.

```
# Link prediction: both engines, every configured rule set
mvn exec:java -Dexec.mainClass="nmRuleExtension.RunExperiment" \
  -Dexec.args="data/NELL995/NELL995.json [full|anyburl|amie]"

# Materialization: arg 2 = rule-confidence percentages
mvn exec:java -Dexec.mainClass="nmRuleExtension.RunMaterialization" \
  -Dexec.args="data/NELL995/NELL995.json 1,5,10"

# Violation counting over a materialization output directory
MAIN=nmRuleExtension.evaluation.MaterializationEvaluationPipeline
mvn exec:java -Dexec.mainClass="$MAIN" \
  -Dexec.args="data/NELL995/NELL995.json <output dir>"
```

The SPARQL queries used to count the constraint violations reported in the paper are listed in the repository `README.md`.

5 Usage of Generative AI

Generative AI (Claude and Claude Code, https://claude.ai/) was used in a supporting fashion for grammar correction and text rephrasing in this work, and in a co-pilot fashion for coding and documentation of the model and the repository.

6 License